



**positive
technologies**



Команда gesti.tech

Задача №2

Получение доступа к прошивке современной компьютерной платы
с использованием различных техник реверс-инжиниринга



О команде

Капитан: Шевцов Сергей Павлович, инженер-электронщик

Кол-во участников: 2 человека

Краткое описание:

Учимся в МГУ им. М.В. Ломоносова и УрФУ им. Б.Н. Ельцина
Команда образовалась после знакомства в ОЦ “Сириус” на профильной программе

Город и регион:

г. Москва
г. Екатеринбург (Свердловская область)



Команда **gesti.tech**

Краткое описание решения

Выявили и проэксплуатировали
11 видов уязвимостей платы на базе
микроконтроллера **RP2040**

Уникальность решения:

Решение сочетает в себе как самые доступные варианты доступа, так и весьма креативные (:

Все описанные способы можно повторить на любом другом подобном устройстве

Команда **gesti.tech**



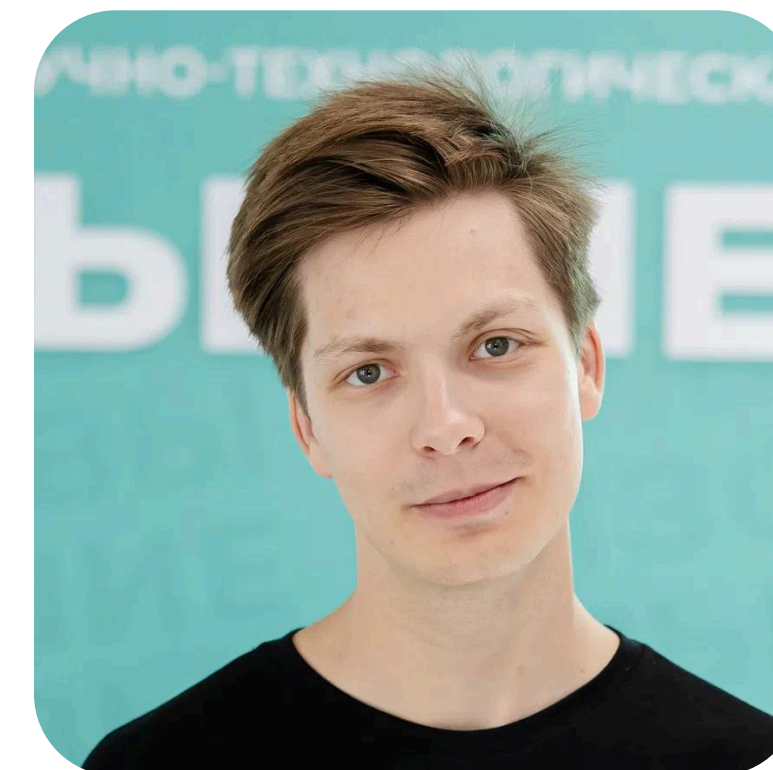
**Сергей
Шевцов**

Инженер-
электронщик

@SergeyShevcov

+7 982 669 6065

УрФУ
им. Б.Н. Ельцина



**Павел
Колесников**

Программист,
аналитик бинарного
кода

@kolezo_pi

+7 912 298 3767

МГУ
им. М.В. Ломоносова

Краткая история команды:

Команда образовалась после знакомства в ОЦ “Сириус” на профильной программе. Имеем большой опыт разработки электронных и программных преоктов.

Почему вы выбрали именно эту задачу из предложенных на хакатоне?

Нас вдохновила сама идея «чёрного ящика»: устройство, которое позиционируется как защищённое хранилище, нужно вскрыть без исходников, схем и документации - только физический доступ и инструменты. Это чистый реверс-инжиниринг в его классическом виде. Кроме того, задача позволяет применить знания по разработке электроники и софта. Любим ковыряться с железяками (:

С какими основными сложностями или вызовами вы столкнулись и как их преодолели?

Одним из самых клёвых моментов - было покопаться в “сыром” бинарнике, понять как он работает и через отладку напрямую управлять работой “камня” микроконтроллера (:

Коротко о решении



Техническая суть решения

Проект представляет полный цикл реверс-инжиниринга CyberSafe: восстановление схемы платы, идентификация RP2040 и микросхемы памяти, реализация 11 работающих векторов атаки всех классов. Используются SWD-отладка через FT232H и OpenOCD/GDB для чтения и записи flash, USB PicoBoot для дампа без вскрытия, прямое подключение к SPI через flashrom, дизассемблирование в Ghidra с SVD и автопереименованием для поиска PIN, криптоанализ XOR-шифрования ФС с извлечением данных без пароля, а также сборка модифицированных UF2-прошивок для обхода блокировки. Каждый метод подтверждён видео, скриптами и бинарными артефактами, что делает решение воспроизводимым и пригодным как методическая база.

Маркетинговая суть решения

Полученная информация о 11 подтверждённых способах взлома позволяет доработать устройство, повысив его защищённость за счёт перехода на MCU с аппаратной защитой чтения и secure boot, хранения хэша PIN вместо открытого значения, привязки шифрования данных к PIN через KDF и добавления анти-брутфорса, что минимизирует вероятность взлома у клиента и сокращает финансовые и репутационные издержки компании. Разработанный проект можно использовать как готовый чек-лист для проверки проектов на уязвимость с конкретными рекомендациями по их устранению, а также как руководство по выбору микроконтроллеров под задачи разного уровня критичности: от RP2040 без защиты до STM32 с RDP Level 2

Брутфорс

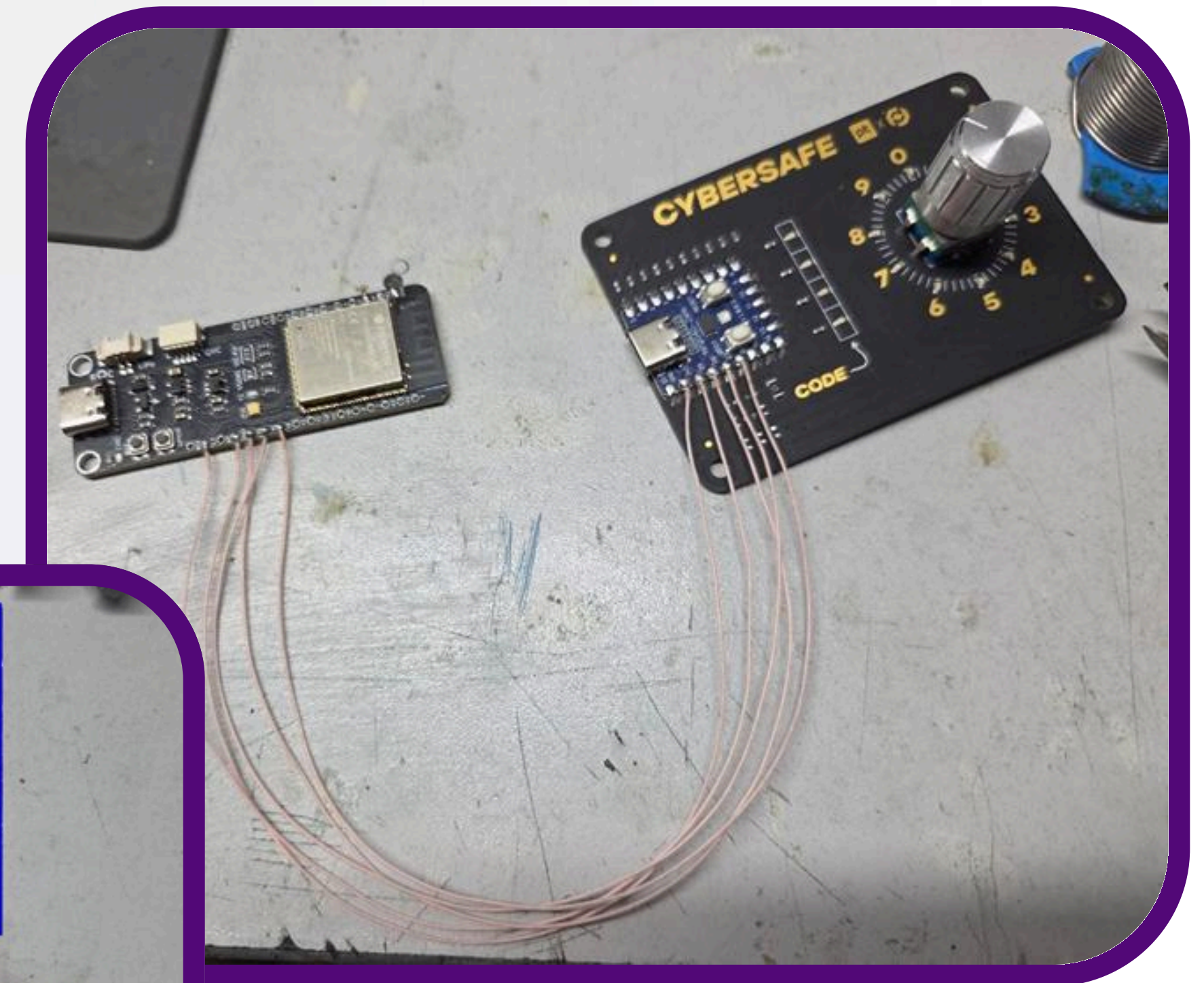
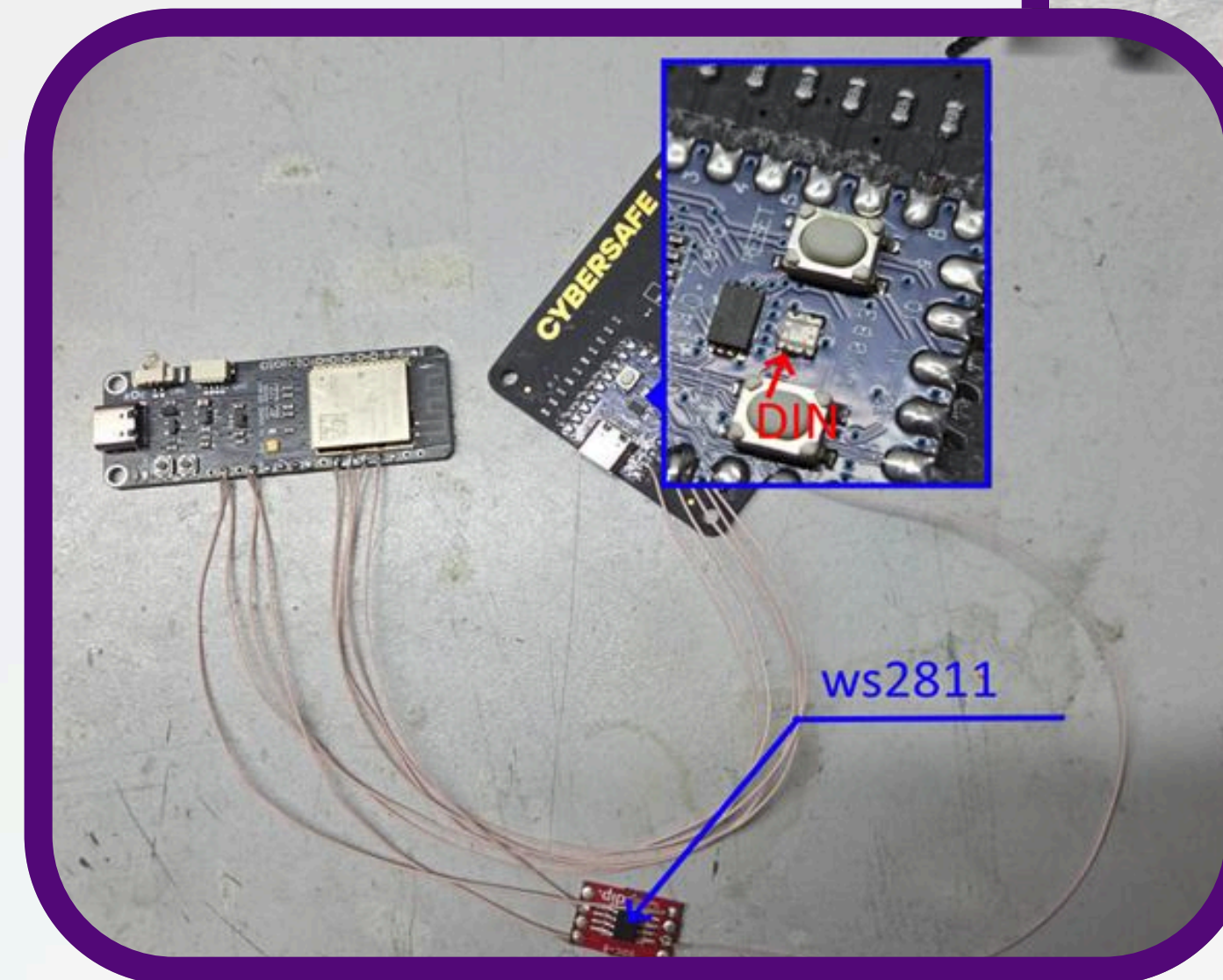
Подключили ESP32 к пинам вместо энкодера
и программно эмулировали ввод PIN-кода

Осциллографом определили предельно возможную
скорость перебора

Рассчётное время перебора ~6 часов

Подобран PIN-код **6061** за 4.5 часа

Останов перебора по сигналу с ws2811,
подключенного к светодиоду
(индикатор корректности PIN-кода)



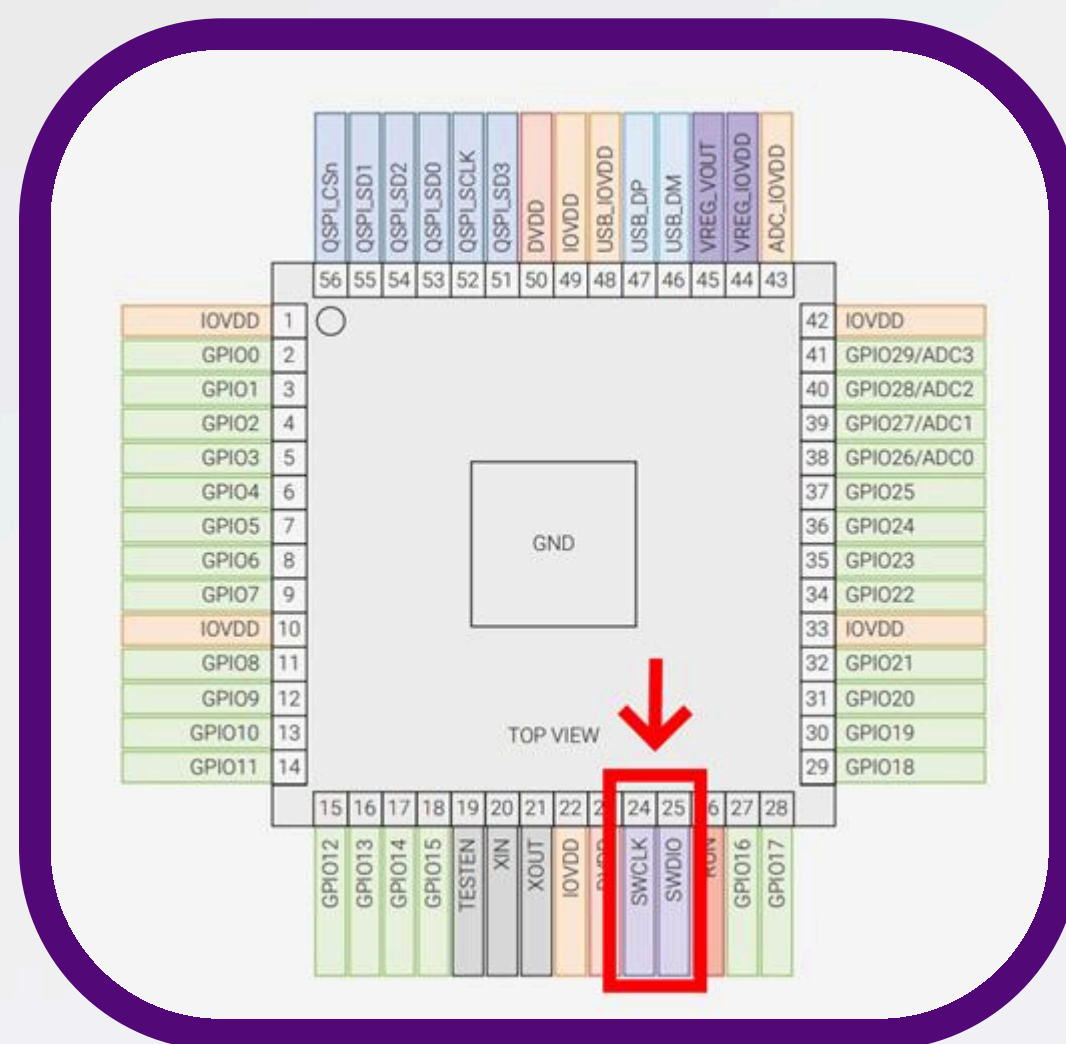
Видео: https://vkvideo.ru/video-241723191_456239025?list=ln-PKEICcR9kCApmB4x2D

Дамп FLASH-памяти, SWD

Напрямую припаялись к пинам SWCLK и SWDIO МК RP2040

Использовали преобразователь FT232H

Сняли дамп памяти размером **2 097 152** байт



Видео: https://vkvideo.ru/video-241723191_456239021?list=In-OURtOFXVXCHYXLf0aM

Подключились к RP2040 через USB интерфейс в режиме PicoBoot

Сняли дамп памяти размером **2 097 152** байт

```
C:\Users\Сергей>picotool save -r 0x10000000 0x10200000 dump.bin
Saving file:      [=====] 100%
Wrote 2097152 bytes to dump.bin
```

2.8.4. USB PICOBOOT Interface

The PICOBOOT interface is a low level USB protocol for interacting with the RP2040 while it is in BOOTSEL mode. This interface may be used concurrently with the USB Mass Storage Interface.

It provides for flexible reading from and writing to RAM or Flash, rebooting, executing code on the device and a handful of other management functions.

Constants and structures related to the interface can be found in the SDK header https://github.com/raspberrypi/pico-sdk/tree/master/src/common/boot_picoboot/include/boot/picoboot.h

2.8.4.1. Identifying The Device

A RP2040 device is recognized by the *Vendor ID* and *Product ID* in its device descriptor (shown in [Table 183](#)).

Field	Value
bLength	18
bDescriptorType	1
bcdUSB	1.10
bDeviceClass	0
bDeviceSubClass	0
bDeviceProtocol	0
bMaxPacketSize0	64
idVendor	0x2e8a
idProduct	0x0003
bcdDevice	1.00
iManufacturer	1
iProduct	2
iSerial	3
bNumConfigurations	1

2.8.4.2. Identifying The Interface

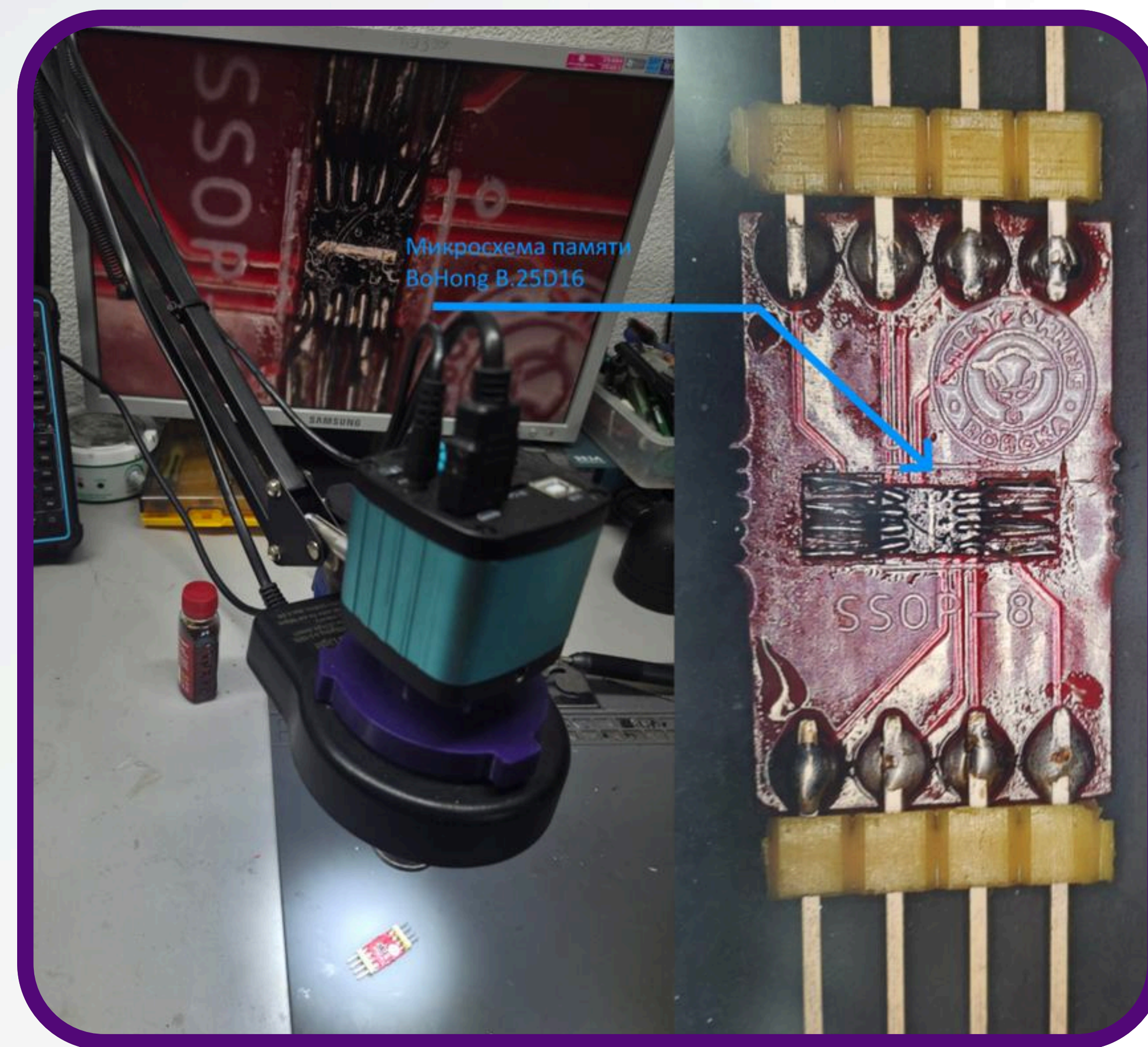
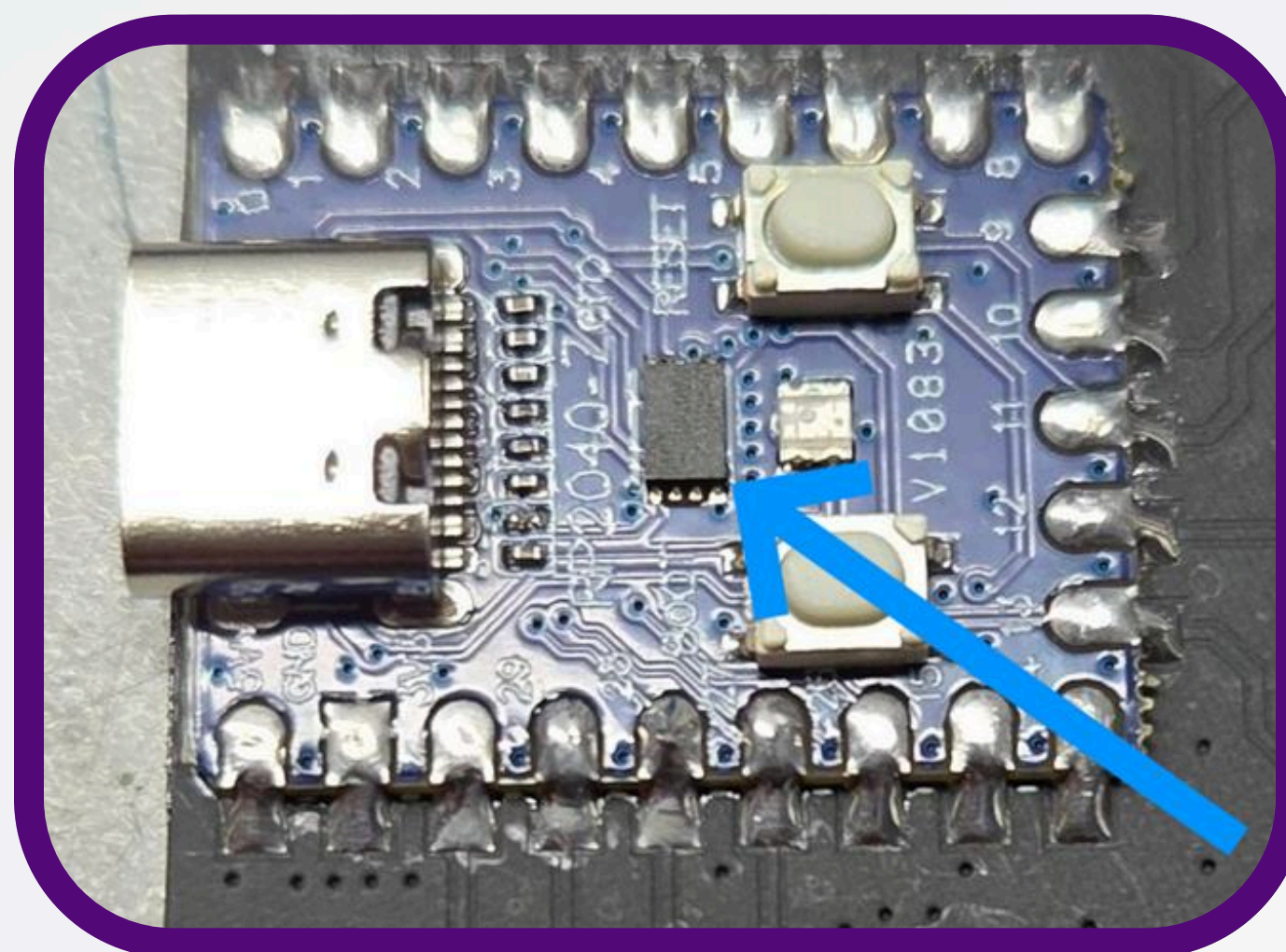
The PICOBOOT interface is recognized by the "Vendor Specific" *Interface Class* and the zero *Interface Sub Class* and *Interface Protocol* (shown in [Table 184](#)). Note that you should not rely on the interface number, as that is dependent on whether the device is also exposing the Mass Storage Interface. Note also that the device equally may not be exposing the PICOBOOT interface at all, so you should not assume it is present.

Видео: https://vkvideo.ru/video-241723191_456239022?list=ln-hTrYQrs1xjZGszWQaC

Дамп FLASH-памяти, SPI

Проволочками напрямую припаялись
к W25Q16JVUXIQ (FLASH-память)

Использовали преобразователь FT232N
Сняли дамп памяти размером **2 097 152** байт



Видео: https://vkvideo.ru/video-241723191_456239026?list=In-sLGcxyahPrr1ZVmjVJ

Анализ дампа FLASH-памяти, PIN-код

Дизасемблирован код микроконтроллера в Ghidra

Определен адрес во FLASH-памяти, где лежит PIN-код:
0x10005500 - 0x10005503 (4 байта) => PIN-код **6061**

Определена структура работы программы

```
10005500 06
```

```
10005501 00
```

```
10005502 06
```

```
10005503 01
```

```
4 void main(void)
5
6 {
7     gpio25_output_init();
8     empty_return_0();
9     lock_ui();
10    tusb_init(0, (char *)0x0);
11    do {
12        tud_task();
13        led_chase();
14    } while( true );
15 }
```

Анализ дампа FLASH-памяти, файловая система (ФС)

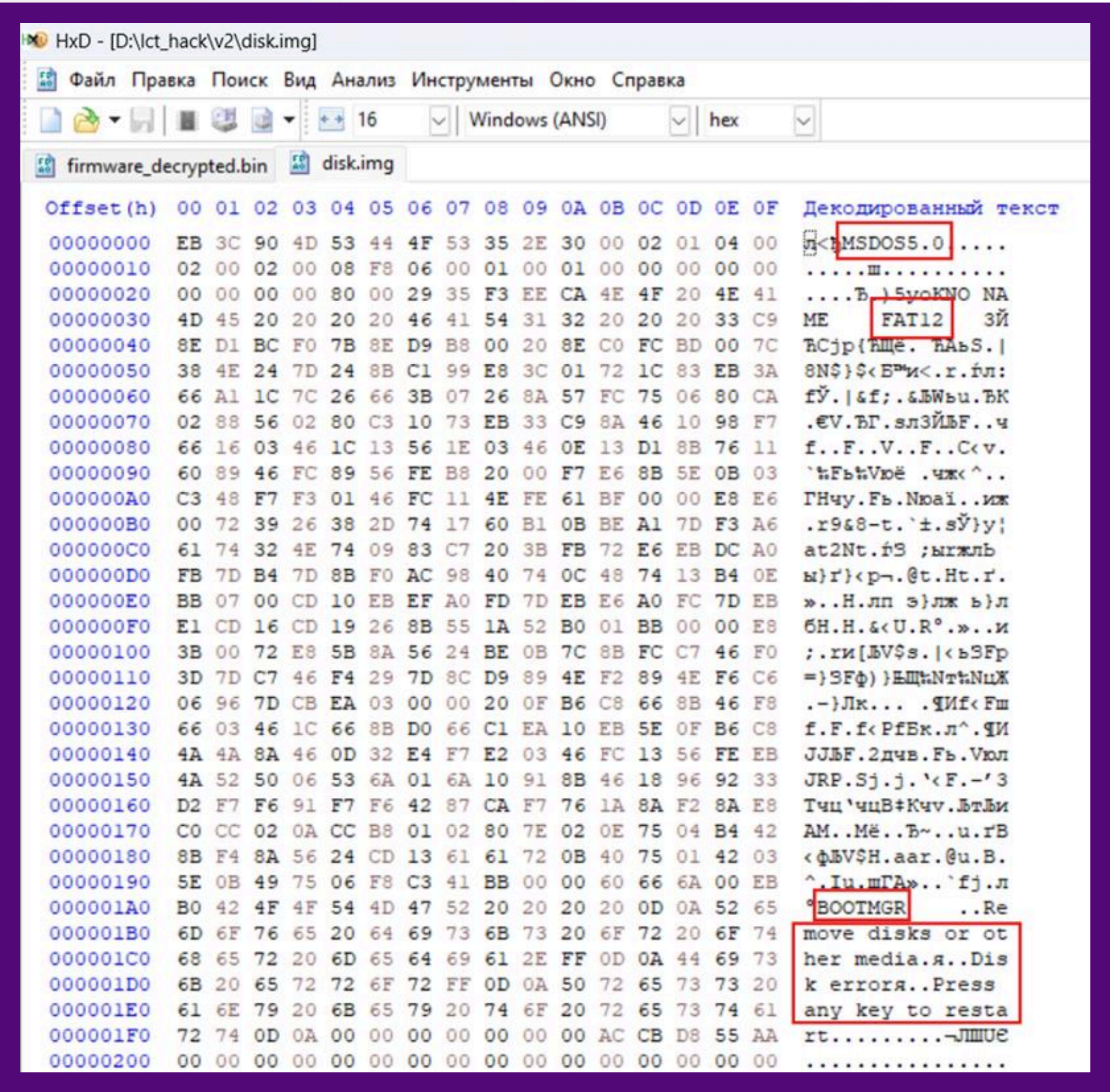
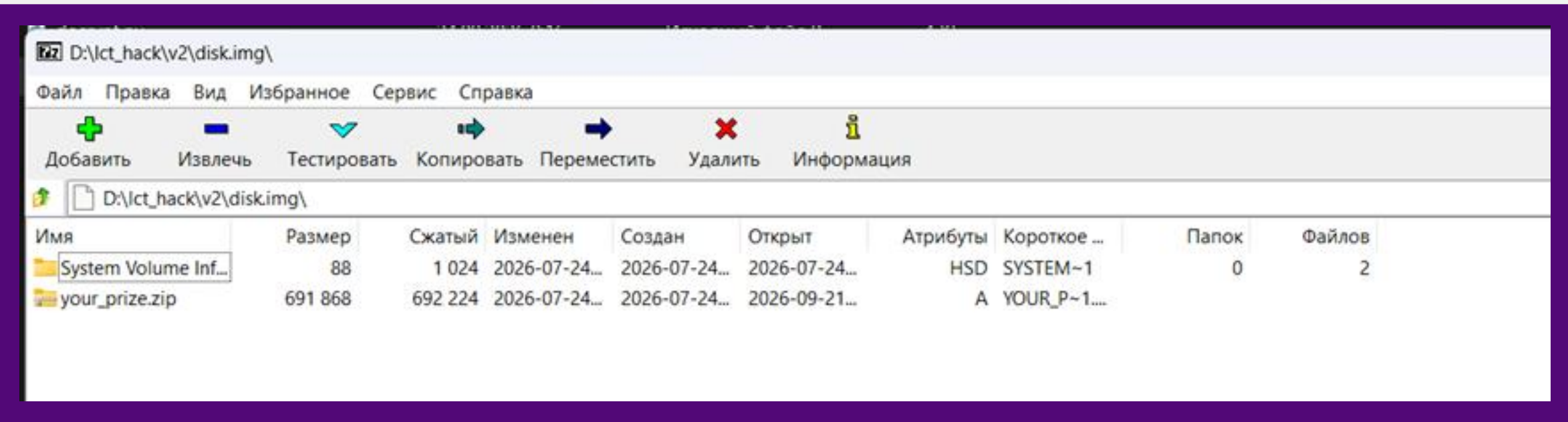
Дизасемблирован код микроконтроллера в Ghidra

Определен адрес во FLASH-памяти,
где лежит ФС:

0x10100000 - 0x101fffff (1 МБ)

Найдена функция в прошивке,
дешифрующая ФС “на лету” (0x10000354)

Написан скрипт на Python,
дешифрующий ФС из дампа



SWD, подмена PIN-кода в RAM

Расставили breakpoint`ы в программе и на работающем микроконтроллере модифицировали RAM, обойдя ввод PIN-кода

```
(gdb) info registers r0 r1 r2 r3 r4 r5 r6 r7
r0          0x12a6908d          312905869
r1          0x0                0
r2          0x0                0
r3          0x6                6
r4          0x1                1
r5          0x100054ff          268457215
r6          0x3                3
r7          0x3                3
(gdb) set $r3 = $r2
(gdb) continue
Continuing.

Breakpoint 7, 0x10000c8a in ?? ()
(gdb) set $r3 = $r2
(gdb) continue
Continuing.

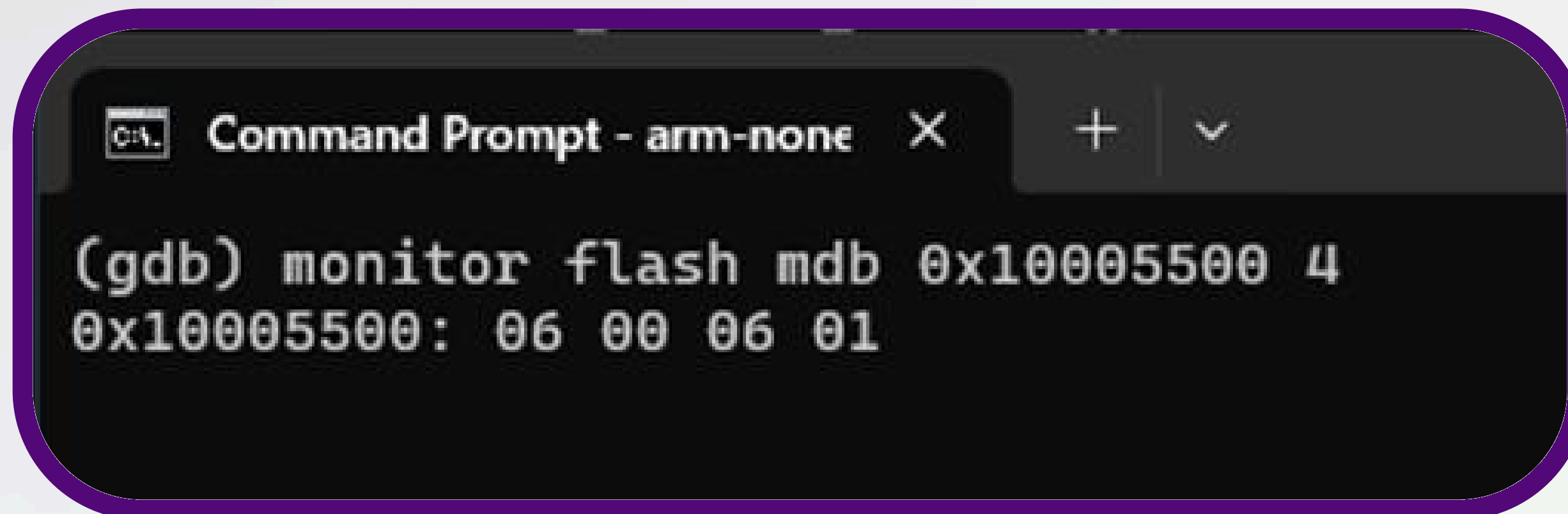
Breakpoint 7, 0x10000c8a in ?? ()
(gdb) set $r3 = $r2
(gdb) continue
Continuing.

Breakpoint 7, 0x10000c8a in ?? ()
(gdb) set $r3 = $r2
(gdb) continue
Continuing.
```

Видео: https://vkvideo.ru/video-241723191_456239017?list=ln-heY3lgcECMfm9b5GLZ

SWD, просмотр PIN-кода в памяти

Одной командой считываем 4 байта FLASH-памяти по адресу 0x10005500 и получаем PIN-код (нужно только знать, какой адрес считывать)



```
Command Prompt - arm-none X + v
(gdb) monitor flash mdb 0x10005500 4
0x10005500: 06 00 06 01
```

Видео: https://vkvideo.ru/video-241723191_456239020?list=ln-rvnjghGljDogOgmA6l

SWD, изменение PIN-кода на 0000

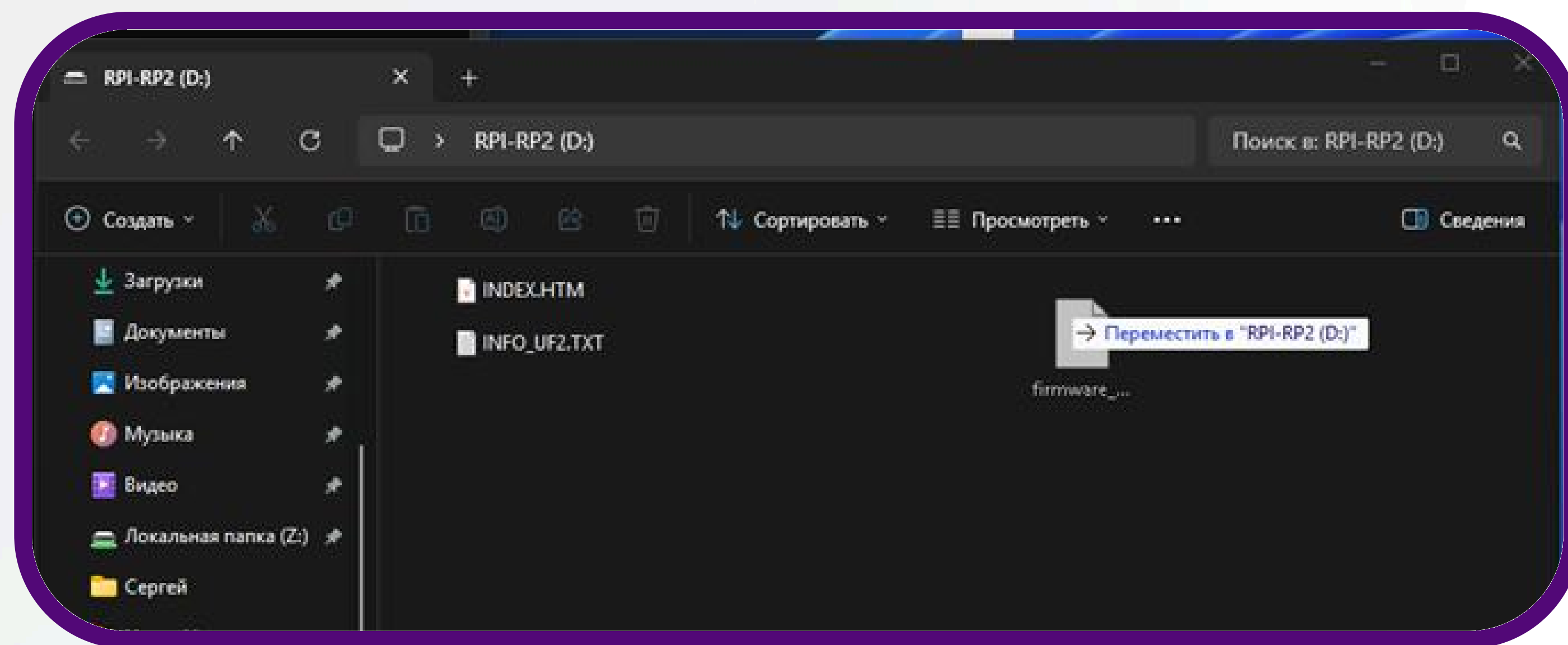
Зануляем 4 байта FLASH-памяти по адресу 0x10005500, устанавливая PIN-код равным **0000**

```
(gdb) monitor flash fillb 0x10005500 0x00 1
Padding at 0x10005501 with 255 bytes (bank write end alignment)
wrote 1 bytes to 0x10005500 in 0.949225s (0.001 KiB/s)
(gdb) monitor flash fillb 0x10005501 0x00 1
Start address 0x10005501 breaks the required alignment of flash bank rp2040.flash
Padding 1 bytes from 0x10005500
Padding at 0x10005502 with 254 bytes (bank write end alignment)
wrote 1 bytes to 0x10005501 in 0.949600s (0.001 KiB/s)
(gdb) monitor flash fillb 0x10005502 0x00 1
Start address 0x10005502 breaks the required alignment of flash bank rp2040.flash
Padding 2 bytes from 0x10005500
Padding at 0x10005503 with 253 bytes (bank write end alignment)
wrote 1 bytes to 0x10005502 in 0.949158s (0.001 KiB/s)
(gdb) monitor flash fillb 0x10005503 0x00 1
Start address 0x10005503 breaks the required alignment of flash bank rp2040.flash
Padding 3 bytes from 0x10005500
Padding at 0x10005504 with 252 bytes (bank write end alignment)
wrote 1 bytes to 0x10005503 in 0.949344s (0.001 KiB/s)
(gdb) monitor flash mdb 0x10005500 4
0x10005500: 00 00 00 00
(gdb) |
```

Видео: https://vkvideo.ru/video-241723191_456239019?list=ln-7dQLw1vw9xWfvHZqki

Модификация прошивки, изменение PIN-кода на 0000

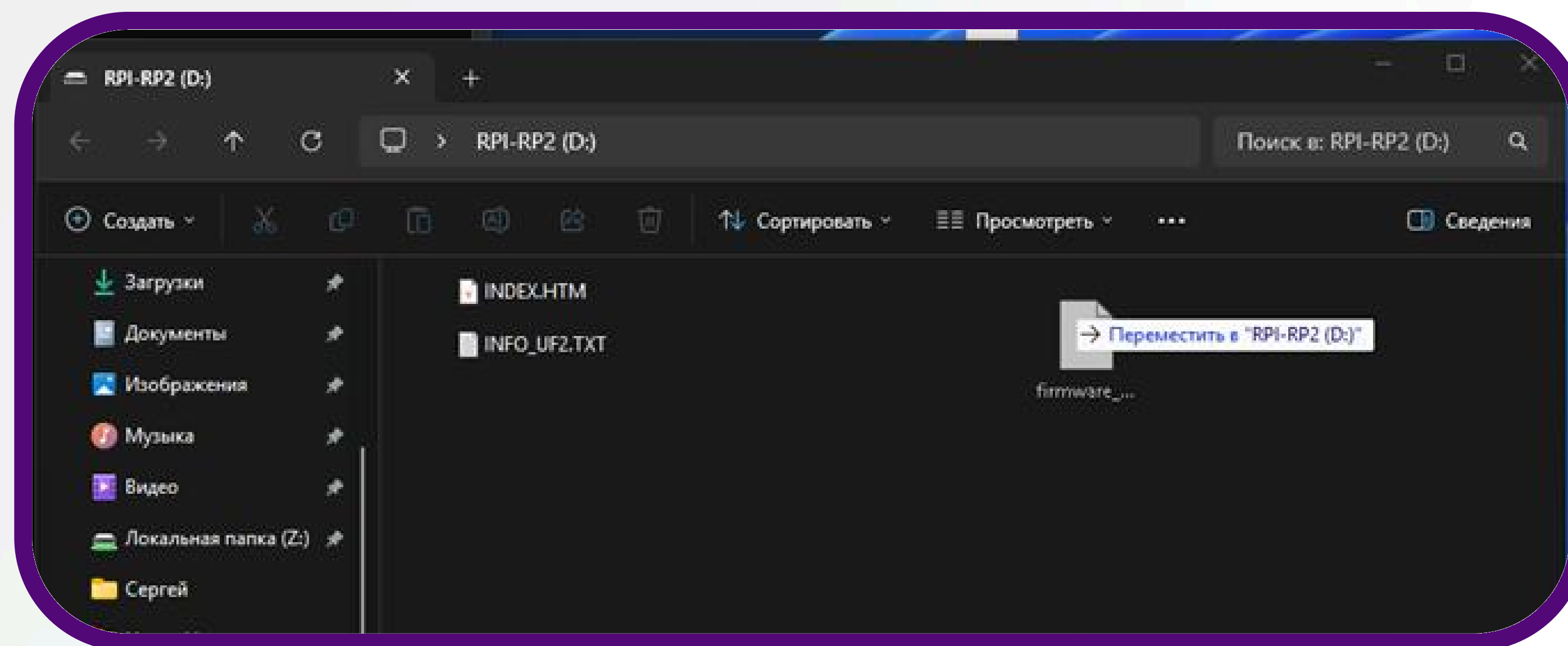
В дампе FLASH-памяти зануляем 4 байта по адресу 0x10005500, устанавливая PIN-код равным **0000**, и загружаем прошивку на микроконтроллер



Видео: https://vkvideo.ru/video-241723191_456239023?list=ln-l7FZaIBJMMuRISb2d1

Модификация прошивки, отключение PIN-кода

В дампе FLASH-памяти убираем вызов `lock_ui()` [0x10000AB0] из функции `main()`, записывая в 0x100008A6 и 0x100008A8 инструкцию NOP



Видео: https://vkvideo.ru/video-241723191_456239024?list=ln-FUeRDYFiFfzGKuLHma

Распаковка 1337 слоёв архивов

1. Получаем любым выше описанным способом **your_prize.zip**
2. Распаковываем **your_prize.zip**
3. Распаковываем **layers.zip**
4. Помещаем в папку **layers** (рядом с **layer_1.zip**) скрипт **extractor.py**
5. Запускаем скрипт, ждём, получаем **prize.html**





**positive
technologies**



Команда gesti.tech

Задача №2

Получение доступа к прошивке современной компьютерной платы
с использованием различных техник реверс-инжиниринга